arXiv:2207.05608v1 [cs.RO] 12 Jul 2022

Inner Monologue: Embodied Reasoning through Planning with Language Models

Wenlong Huang†, Fei Xia†, Ted Xiao†, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Luu, Sergey Levine, Karol Hausman, Brian Ichter
Robotics at Google, † equal contribution and alphabetically listed
Project website: https://innermonologue.github.io

Abstract: Recent works have shown how the reasoning capabilities of Large Language Models (LLMs) can be applied to domains beyond natural language processing, such as planning and interaction for robots. These embodied problems require an agent to understand many semantic aspects of the world: the repertoire of skills available, how these skills influence the world, and how changes to the world map back to the language. LLMs planning in embodied environments need to consider not just what skills to do, but also how and when to do them - answers that change over time in response to the agent's own choices. In this work, we investigate to what extent LLMs used in such embodied contexts can reason over sources of feedback provided through natural language, without any additional training. We propose that by leveraging environment feedback, LLMs are able to form an *inner monologue* that allows them to more richly process and plan in robotic control scenarios. We investigate a variety of sources of feedback, such as success detection, scene description, and human interaction. We find that closed-loop language feedback significantly improves high-level instruction completion on three domains, including simulated and real table top rearrangement tasks and long-horizon mobile manipulation tasks in a kitchen environment in the real world.

1 Introduction

Intelligent and flexible embodied interaction requires robots to be able to deploy large repertoires of basic behaviors in appropriate ways, sequence these behaviors as needed for long horizon tasks, and also recognize when to switch to a different approach if a particular behavior or plan is unsuccessful. High-level planning, perceptual feedback, and low-level control are just a few of the sub-tasks that would need to be seamlessly combined together to perform the sort of reasoning required for an embodied agent, such as a robot, to intelligently act in the world. While conventionally these challenges have been approached from the perspective of planning (e.g., TAMP [1]) or hierarchical learning (e.g., HRL [2]), effective high-level reasoning about complex tasks also requires semantic knowledge and understanding of the world.

One of the remarkable observations in recent machine learning research is that large language models (LLMs) can not only generate fluent textual descriptions, but also appear to have rich internalized knowledge about the world [3, 4, 5, 6, 7]. When appropriately conditioned (e.g., prompted), they can even carry out some degree of deduction and respond to questions that appear to require reasoning and inference [8, 9, 10, 11, 12, 13]. This raises an intriguing possibility: beyond their ability to interpret natural language instructions, can language models further serve as reasoning models that combine multiple sources of feedback and become interactive problem solvers for embodied tasks, such as robotic manipulation?

Prior studies show that language helps humans internalize our knowledge and perform complex relational reasoning through *thinking in language* [14, 15, 16, 17, 18]. Imagine the "inner monologue" that happens when a person tries to solve some task: "I have to unlock the door; let me try to pick up the key and put it in the lock... no, wait, it doesn't fit, I'll try another one... that one worked, now I can turn the key." The thought process in this case involves choices about the best immediate action to solve the high-level task ("pick up the key"), observations about the outcomes of attempted actions ("it doesn't fit"), and corrective actions that are taken in response to these observations ("I'll try another one"). Inspired by the human thought process, we propose that such an inner monologue is a natural framework for incorporating feedback for LLMs.

Our work studies these questions by combining LLMs with various sources of textual feedback, only utilizing few-shot prompting without any additional training. We observe that similarly to recent work [19], natural language provides a universal and interpretable interface for such grounding of model communication and allows them to incorporate their conclusions in an overarching inner monologue driven by a language model. While prior work has investigated using language models as planners [20, 21] or incorporating

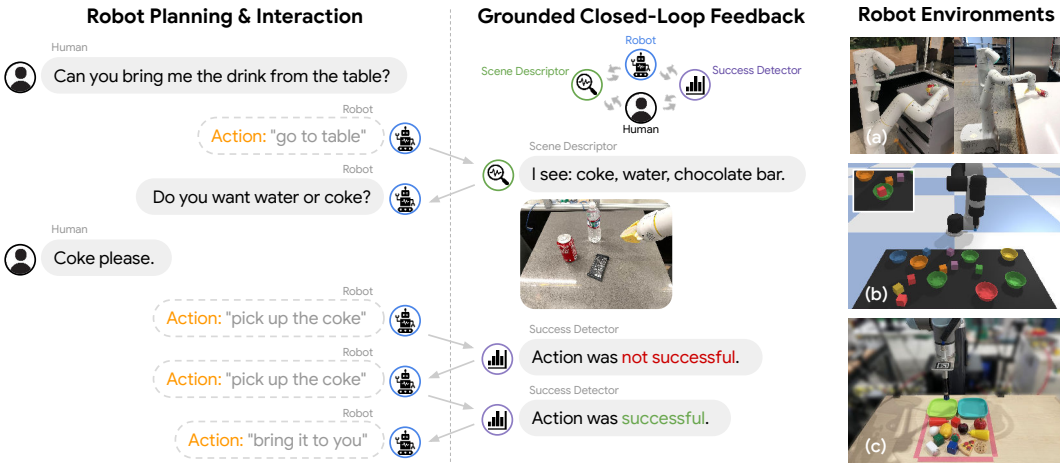


Figure 1: Inner Monologue enables grounded closed-loop feedback for robot planning with large language models by leveraging a collection of perception models (e.g., scene descriptors and success detectors) in tandem with pretrained language-conditioned robot skills. Experiments show our system can reason and replan to accomplish complex long-horizon tasks for (a) mobile manipulation and (b,c) tabletop manipulation in both simulated and real settings.

multimodal-informed perception through language [19], to the best of our knowledge no work has studied the critical link of not only planning with language, but also informing *embodied feedback with language*, which we investigate in this work.

Specifically, we study methods and sources of feedback for closing the agent-environment loop via an inner monologue and their impact on downstream execution success and new capabilities arising from such interaction. In particular, we combine multiple perception models that perform various tasks such as language-conditioned semantic classification or language-based scene description, together with feedback provided by a human user that the robot is cooperating with. To execute the commands given by a user, the actions are chosen from a set of pre-trained robotic manipulation skills together with their textual descriptions that can be invoked by a language model. Our proposed system Inner Monologue chains together these various components (perception models, robotic skills, and human feedback) in a shared language prompt, enabling it to successfully perform user instructions.

Finally, we show that Inner Monologue, without requiring additional training beyond a frozen language model and pre-trained robotic skills, can accomplish complex, long-horizon, and unseen tasks in simulation as well as on two real-world robotic platforms. Notably, we show that it can efficiently retry under observed stochastic failure, replan under systematic infeasibility, or request human feedback for ambiguous queries, resulting in significantly improved performance in dynamical environments. As a demonstration of the versatility of LLMs and grounded closed-loop feedback, we additionally show several surprising capabilities emerging from the inner monologue formulation, including continued adaptation to new instructions, self-proposed goals, interactive scene understanding, multilingual interactions, and more.

2 Related Work

Task and Motion Planning. Task and motion planning [22, 23] requires simultaneously solving a high-level, discrete task planning problem [24, 25, 26], and a low-level, continuous motion planning problem [27]. Traditionally, this problem has been solved through optimization [28, 29] or symbolic reasoning [24, 26], but more recently machine learning has been applied to aspects of the problem via learned representations, learned task-primitives, and more [30, 31, 32, 33, 34, 35, 36, 37, 38]. Some works utilize language for planning and grounding [39, 40, 41, 42, 43, 44]. Others have approached the problem through hierarchical learning [45, 46, 34, 47, 48, 49, 50]. In this work, we leverage pre-trained LLMs and their semantic knowledge, along with trained low-level skills, to find feasible plans.

Task Planning with Language Models. Various prior works have explored using language as a space for planning [51, 52, 20, 53, 21, 19]. Similar to ours are recent task planning approaches that leverage pre-trained autoregressive LLMs to decompose abstract, high-level instructions into a sequence of low-level steps executable by an agent [20, 21] in a zero-shot manner. Specifically, Huang et al. [20] prompt GPT-3 [9] and Codex [54] to generate action plans for embodied agents, where each action step is semantically translated to an admissible action with a Sentence-RoBERTa model [55, 56]. SayCan [21] instead grounds the actions by mul-

tiplying each candidate action's probability under FLAN [57] with the action's value function, which serves as a proxy for affordance [34]. However, both approaches effectively produce the plan while assuming that each proposed step is executed successfully by the agent. As a result, these approaches may not be robust in handling intermediate failures in dynamic environments or with poor lower level policies. We explore in Inner Monologue ways to incorporate grounded feedback from the environment into the LLM as we produce each step in the plan.

Fusing Vision, Language, and Control in Robotics. Various works have investigated strategies for the challenging problem of fusing vision, language, and control [58, 59, 60, 61, 62, 63, 64]. While pretrained LLMs typically train only on text data, pretrained visual-language models (e.g., CLIP [65]) are trained on joint image(s) and corresponding text captions via variants of masked language modeling (MLM) objective [66, 67, 68, 69], contrastive loss [70, 71, 65] or other supervised objectives[72, 73]. CLIP has been employed in several robotics and embodied settings in zero-shot manner [74], or combined with Transporter networks [75] as in CLIPort [76]. Socratic Models [19] combines several foundation models (e.g., GPT-3 [9], ViLD [77]) and language-conditioned policies, using language as the common interface, and demonstrated manipulating objects in a simulated vision-based robotic manipulation environment.

3 Leveraging Embodied Language Feedback with Inner Monologue

In this section, we introduce how LLMs can act as interactive problem solvers and incorporate embodied environment observations into grounded planning through a process we refer to as Inner Monologue.

3.1 Problem Statement

Our setting consists of an embodied robotic agent attempting to perform a high-level natural language instruction i. This robotic agent is only capable of executing short-horizon skills from a library of previously trained policies $\pi_k \in \Pi$ with short language descriptions ℓ_k, which may be trained with reinforcement learning or behavioral cloning. The "planner," which is a pretrained LLM [20, 21], attempts to find a sequence of skills to accomplish the instruction. To observe the environment, the planner has access to textual feedback o from the environment that can be appended to the instruction or requested by the planner. The observation o may be success detection, object detection, scene description, visual-question answering, or even human feedback. Our work studies to what extent the LLM planner is able to reason over and utilize such feedback to "close the loop" with the environment and improve planning.

3.2 Inner Monologue

We formulate an "inner monologue" by continually injecting information from the various sources of feedback into the LLM planning language prompts as the robot interacts with the environment. While LLMs have demonstrated exceptional planning capabilities for embodied control tasks [20], prior works have found it crucial to ground LLM predictions with external components such as affordance functions [21] in order to produce useful plans that are executable by robots. However, LLMs used in this context have thus far remained one-directional – providing a list of skills, without making corrections or leveraging opportunities to replan accordingly. In contrast, Inner Monologue studies settings where grounded environment feedback is provided directly to the LLM in a closed-loop fashion. This promotes improved LLM reasoning in complex long-horizon settings, even before any external affordance-based grounding methods are applied.

Our analysis assumes textual feedback is provided to the planner, but does not assume a single specific method of fusing LLM planning with low-level robotic control or a specific method of extracting environment feedback into language. Rather than focusing on a particular algorithmic implementation, our aim is to provide a case study on the value of incorporating different types of feedback into closed-loop LLM-based planning. Thus, Inner Monologue in Sec 4 utilizes language feedback within separate systems that incorporate different LLMs, different methods of fusing planning with control, different environments and tasks, and different methods of acquiring control policies. We note that in our specific implementations of Inner Monologue, we use pre-trained LLMs for planning that are not finetuned, but rather evaluated solely with few-shot prompting; the full prompts can be found in the Appendix.

3.3 Sources of Feedback

In theory any type of environment feedback can inform the LLM planner, as long as it can be expressed through language. We focus on the specific forms of feedback shown in Fig 2, which can be broken down into task-specific feedback, such as success detection, and scene-specific feedback (either "passive" or "active"),

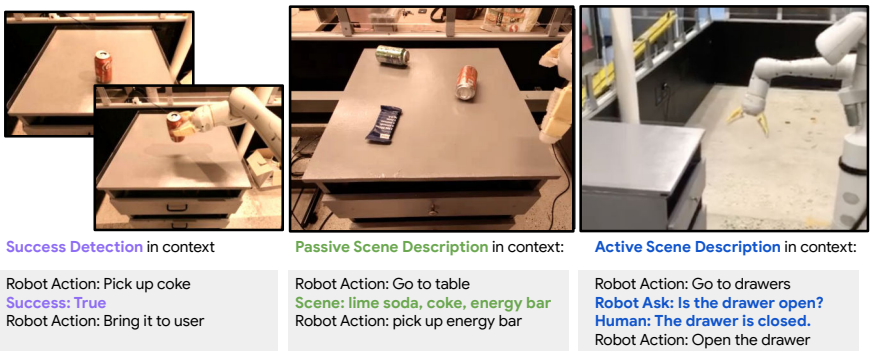


Figure 2: Various types of textual feedback. **Success Detection** gives task-specific task completion information, **Passive Scene Description** gives structured semantic scene information at every planning step, and **Active Scene Description** gives unstructured semantic information only when queried by the LLM planner.

which describes the scene. Specific instantiations and implementation details of each type of feedback can be found in Sec 4.1, Sec 4.2, and Sec 4.3 respectively for each domain.

Success Detection. Semantic success detection is a binary classification problem of whether the low-level skill π_k has succeeded. Engineered success detectors can operate on ground-truth state in simulation, while learned success detectors can be trained on real examples of successes and failures in the real world [78, 79, 80, 81, 82]. We use the output of success detectors in language form, which we refer to as *Success* feedback.

Passive Scene Description. While there are many ways to describe the semantics contained within a scene, we use the term Passive Scene Description to broadly describe sources of scene feedback that are consistently provided and follow some structure. Passive Scene Description covers all sources of environment grounding feedback that are automatically provided and injected into the LLM prompt without any active prompting or querying by the LLM planner. One common type of such feedback is object recognition [83, 84, 85, 86] – we refer to the textual outputs of such object recognizers as *Object* feedback. We also demonstrate the use of a task-progress scene description in the simulated tabletop rearrangement environment, to which we refer as *Scene* feedback.

Active Scene Description. As the proactive counterpart to Passive Scene Description, Active Scene Description encompasses sources of feedback that are provided directly in response to active queries by the LLM planner. In this case, the LLM can directly ask a question about the scene, and this question can be answered either by a person, or by another pretrained model, such as a Visual Question Answering (VQA) model [87, 88, 89, 90]. While the previous types of feedback are strictly structured and narrow in their scope, in the Active Scene Description setting the LLM can receive unstructured answers to open-ended questions, allowing it to actively gather information relevant to the scene, the task, or even preferences of the user (in the case of human-provided response). The combined output we send to the LLM planner includes both the LLM-generated question along with the response. As we aim to investigate *whether* and *how* a LLM planner can incorporate such feedback and wish to study both structured VQA-style human feedback as well as unstructured human preferences feedback, we only consider human-provided response in this work, which we refer to as *Human* feedback.

4 Experimental Results

In order to study how different sources of environment feedback can support a rich inner monologue that enables complex robotic control, we analyze diverse long-horizon manipulation and navigation tasks in simulation and in the real world. As Inner Monologue is not dependent on a specific LLM or a type of grounding feedback, we study different Inner Monologue implementations in three environments with different LLM planning methods and different sources of feedback from the environment. Below, we show results for a tabletop manipulation environment in sim (Sec 4.1) and real (Sec 4.2) as well as a mobile manipulation environment in real (Sec 4.3). For more details about the experiment setup and results, please refer to the Appendix.

4.1 Simulated Tabletop Rearrangement

We experiment with vision-based block manipulation tasks in a Ravens-based [75] simulation environment to evaluate our method against several baselines and ablate across varying amounts of textual feedback. Given

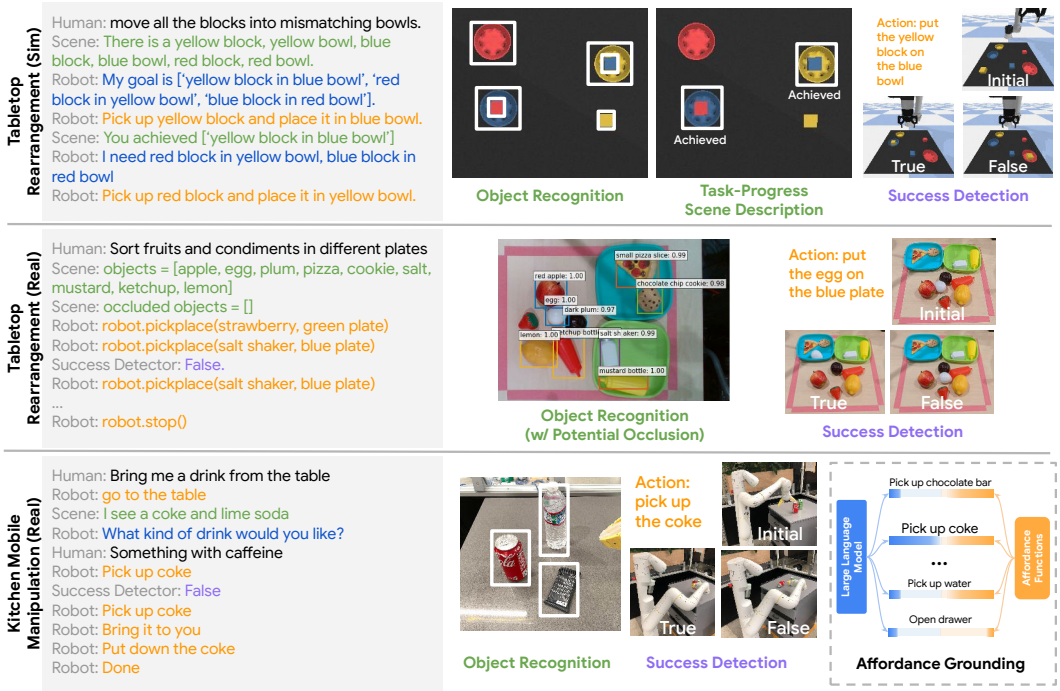


Figure 3: Different instantiations of Inner Monologue in three distinct domains – simulated tabletop rearrangement (*top*), real-world tabletop rearrangement (*middle*), and real-world kitchen mobile manipulation (*bottom*). Each domain uses different prompts and different feedback models. Sharing across the domains is the same Inner Monologue formulation that uses a pre-trained langauge model to take in a **human instruction** and decompose it into a sequence of **actionable steps** by the agent, while accounting for injected embodied feedback from different models, such as **object recognizers** and **success detectors**. In real-world kitchen mobile manipulation domain (*bottom*), we additionally ground the actions using pre-trained affordance functions built in [21], which do not communicate back to the language model.

a number of blocks and bowls on a table, a robotic arm containing a gripper is tasked with rearranging these objects in some desired configuration, specified by natural language (e.g., "putting the blocks in the bowls with matching colors"). We evaluate each method on four seen tasks and four unseen tasks, where seen tasks may be used for training (in the case of supervised baseline) or used as few-shot prompting for LLM planner.

This instantiation of Inner Monologue uses (i) InstructGPT [9, 91] as the LLM for multi-step planning [20, 21], (ii) scripted modules to provide language feedback in the form of object recognition (*Object*), success detection (*Success*), and task-orogress scene description (*Scene*), and (iii) a pre-trained language-conditioned pick-and-place primitive (similar to CLIPort [76] and Transporter Nets [75]). *Object* feedback informs the LLM planner about the objects present in the scene, and the variant using only *Object* feedback is similar to the demonstrated example in [19] in this environment. *Success* feedback informs the planner about success/failure of the most recent action. However, in the presence of many objects and test-time disturbances, the complex combinatorial state space requires the planner to additionally reason about the overall task progress (e.g., if the goal is to stack multiple blocks, the unfinished tower of blocks may be knocked over by the robot). Therefore, task-progress scene description (*Scene*) describes the semantic sub-goals inferred by the LLM towards completing the high-level instruction that are achieved by the agent so far. For the variant that uses *Object* + *Scene* feedback, due to the additional reasoning complexity, we find that adding chain-of-thought [10, 12, 13] can improve the consistency between inferred goals and achieved goals.

We additionally compare to a multi-task CLIPort policy directly trained on long-horizon task instructions (i.e., without using LLM for planning). Because CLIPort is a single-step policy and does not terminate spontaneously during policy rollout, we report CLIPort evaluations with oracle termination (i.e., repeat until oracle indicates task completion) and fixed-step termination (i.e., repeat for k steps). While Inner Monologue terminates when the LLM stops generating new steps, we similarly set the maximum number of steps to be k for practical considerations. We use $k = 15$. To simulate real-world disturbances and evaluate the system's robustness to disturbances, we add Gaussian noise to multiple levels of the system at test time: $\mathcal{N}(0,3)$ for pixel observation, $\mathcal{N}(0,2.5)$ for policy primitive (i.e., pick-place pixel heatmaps), $\mathcal{N}(0,0.02m)$ for place locations.

Results are shown in Table 1, and example prompt can be found in Fig 3.

	Tasks	CLIPort	+oracle	+LLM	+Inner Monologue	
				Object	*Object + Success*	*Object + Scene*
Seen Tasks	"Pick and place"	24.0%	74.0%	80.0%	90.0%	**94.0%**
	"Stack all the blocks"	2.0%	32.0%	4.0%	10.0%	**26.0%**
	"Put all the blocks on the [x] corner/side"	2.0%	32.0%	**30.0%**	28.0%	**30.0%**
	"Put all the blocks in the [x] bowl"	32.0%	94.0%	52.0%	46.0%	**56.0%**
Unseen Tasks	"Put all the blocks in different corners"	0.0%	0.0%	20.0%	20.0%	**26.0%**
	"Put the blocks in their matching bowls"	0.0%	0.0%	56.0%	70.0%	**82.0%**
	"Put the blocks on mismatched bowls"	0.0%	0.0%	62.0%	76.0%	**86.0%**
	"Stack all the blocks on the [x] corner/side"	0.0%	0.0%	0.0%	4.0%	**6.0%**

Table 1: Success rates for various methods, averaged across 50 episodes in Ravens-based environment with test-time disturbances. CLIPort + oracle indicates that CLIPort was provided a "termination" oracle. Although CLIPort can receive visual feedback from the environment, we show that LLM-informed feedback can effectively enable the planner to retry/replan in the presence of failures, while enjoying the generalization benefits of LLMs to unseen tasks.

Analysis. As shown in Table 1, all variants of Inner Monologue perform well on seen tasks, and by leveraging rich semantic knowledge in the pre-trained LLM, the performance can be directly translated to unseen tasks without further training. Furthermore, aligned with our hypothesis, Inner Monologue with *Object + Scene* performs the best because of its ability to keep track of all goal conditions and currently achieved goals. Finally, we observe that non-hierarchical and solitary systems such as CLIPort (i) struggle at generalizing to unseen long-horizon tasks under test-time disturbances, and (ii) on training tasks, an oracle is also often required to indicate task completion for good performance.

4.2 Real-World Tabletop Rearrangement

We evaluate Inner Monologue on a real-world robot platform designed to resemble the simulation experiments presented in Sec 4.1 using motion primitives for tabletop pick and place. The setup consists of a UR5e robot arm equipped with a wrist-mounted Intel RealSense RGB-D camera overlooking a workspace of diverse objects – from toy blocks to food items to condiments (shown in Fig 3). We use an instantiation of Inner Monologue using (i) InstructGPT [9, 91] as the LLM for multi-step planning, (ii) a pretrained open-vocabulary object recognition with MDETR [92] to generate a list of currently visible objects and list of previously visible objects that are no longer visible (*Object*), (iii) heuristics on the object bounding box predictions from MDETR for Success Detection (*Success*), and (iv) a zero-shot pick and place policy that uses an LLM to parse target objects from a language command (e.g., given by the planner) and then executes scripted suction-based picking and placing primitives at the center of the target objects' bounding boxes. Aside from the pretraining of the LLM and MDETR (which are available out-of-the-box), the system does not require any model finetuning to perform pick and place tasks with new objects.

We investigate two tasks: (i) a simple 3-block stacking task where 2 blocks are already pre-stacked, and (ii) a more complex long-horizon sorting task to place food in one plate and condiments in another (where categorizing food versus condiments is autonomously done by the LLM planner). Since default pick and place performance is generally quite high on the system, we artificially inject Gaussian noise into the policy actions (i.e., add standard deviation σ=4mm clipped at 2σ) to stress test recovery from failures via replanning with grounded closed-loop feedback. Results are presented in Table 2. Note that the system is also subject to noisy object and success detections due to the additional challenge of real-world perception and clutter.

	LLM	+Inner Monologue		
Task Family	*Object*	*Object*	*Success*	*Object + Success*
Finish 3-block stacking	20%	40%	40%	**100%**
Sort fruits from bottles	20%	50%	40%	**80%**
Total	20%	45%	40%	**90%**

Table 2: Inner Monologue (with object recognition and success detection feedback) on a real pick and place robot exceeds the performance of baseline alternatives, as measured by average task success rates over 10 runs. For both tasks, we observe significant improvement in Inner Monologue compared to the baseline, with the two embodied feedback being complementary to each other.

Analysis. We compare different variants of Inner Monologue with different LLM-informed closed-loop feedback, as well as an open-loop variant that only runs object recognition once at the beginning of the task (similar to the system demonstrated in [19]). The partial 3-block stacking task highlights an immediate failure mode of this baseline, where the initial scene description struggles to capture a complete representation of the

scene (due to clutter and occlusion) to provide as input to the multi-step planner. As a result, the system only executes one pick and place action – and cannot recover from mistakes. To address these shortcomings, Inner Monologue (*Object + Success*) leverages closed-loop scene description and success detection after each step, which allows it to successfully replan and recover from policy mistakes.

Additional ablations with Inner Monologue also show that (i) common failures induced by lack of closed-loop scene description are largely due to initially occluded objects not being part of the LLM generated plans, and (ii) failures induced by lack of success detection come from not retrying pick and place actions that have failed due to policy noise. Overall, we observe that both components are complementary and important in maintaining robust recovery modes for real rearrangement tasks.

4.3 Real-World Mobile Manipulator in a Kitchen Setting

We implement Inner Monologue in a robotic system using the kitchen environment and task definitions described in SayCan [21]. The Everyday Robots robot, a mobile manipulator with RGB observations, is placed in an office kitchen to interact with common objects using concurrent [93] continuous closed-loop control.

The baseline, SayCan [21], is a method that plans and acts in diverse real world scenarios by combining an LLM with value functions of underlying control policies. While SayCan creates plans that are grounded by the affordances of value functions, the LLM predictions in isolation are never given any closed-loop feedback.

We use an instantiation of Inner Monologue that uses (i) PALM [8] as the LLM for multi-step planning, (ii) value functions from pre-trained control policies for affordance grounding [21], (iii) a learned visual classification model for success detection feedback (*Success*), (iv) human-provided object recognition feedback (*Object*), and (v) pre-trained control policies for relevant skills in the scene. We also perform a case study where we allow the agent to ask questions and source *Human* feedback directly; results are shown in Fig 5a and the Appendix.

We evaluate methods on 120 evaluations over three task families: 1) four manipulation tasks, 2) two dexterous manipulation tasks utilizing drawers, and 3) two long-horizon combined manipulation and navigation tasks. In order to better study how Inner Monologue improves reasoning in especially challenging scenarios, we consider an experiment variant where we add adversarial disturbances during control policy executions that cause skill policy rollouts to fail. These disturbances may be fairly simple and just require the policy to try again, or it may complex enough that the policy needs to replan and select a completely new skill. While these failures occur naturally even without perturbances, the adversarial disturbances creates a consistent comparison between methods that requires retrying or replanning to accomplish the original instruction.

		+Inner Monologue	
Task Family	**SayCan**	*Success*	*Object + Success*
No Disturbances			
Manipulation	50.0%	62.5%	**75.0%**
Mobile Manipulation	50.0%	50.0%	**75.0%**
Drawers	83.3%	83.3%	**100.0%**
With Disturbances			
Manipulation	12.5%	25.0%	**33.3%**
Mobile Manipulation	0.0%	25.0%	**75.0%**
Drawers	0.0%	**44.4%**	**44.4%**
Total	30.8%	48.7%	**60.4%**

Table 3: Averaged success rate across 120 evaluations on several task families in our real-world mobile manipulation environment. We consider a standard setting and adversarial setting with external human disturbances. In all cases, LLM-informed embodied feedback is shown to be effective in improving robustness of the system, especially when low-level policies are prone to failures.

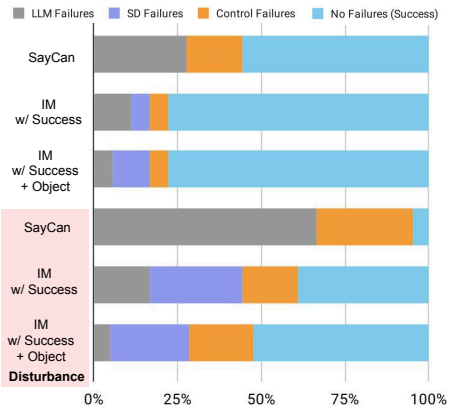
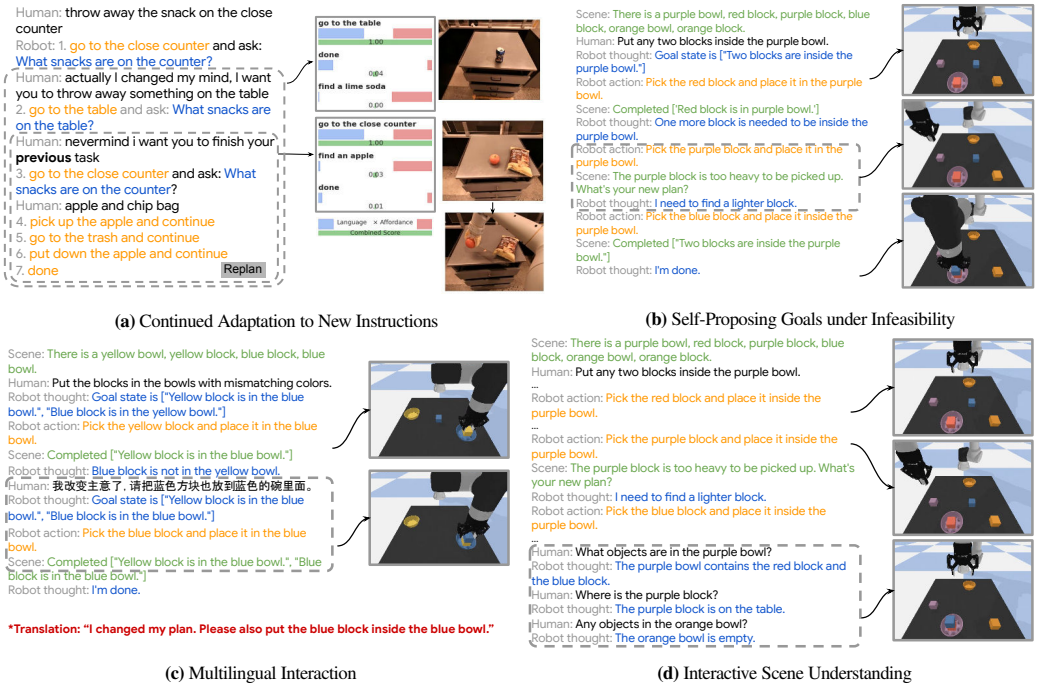


Figure 4: Failure causes on 120 evaluations. When disturbances are added (red), only the Inner Monologue variants consistently complete the instructions.

Analysis. The results of real robot experiments are shown in Table 3. Without adversarial disturbances, the baseline method SayCan performs reasonably on all three task families, yet incorporating LLM-informed feedback on skill success/failure and presence of objects allows the Inner Monologue to effectively retry or replan under natural failures, providing further improvement to the baseline. The most notable difference is in the cases with adversarial disturbances when a policy failure is forced to occur. Without any LLM-informed feedback SayCan has success rate close to 0% since it is does not have explicitly high-level retry behavior. Inner Monologue significantly outperforms SayCan because of its ability to invoke appropriate recovery



(a) Continued Adaptation to New Instructions

(b) Self-Proposing Goals under Infeasibility

(c) Multilingual Interaction

(d) Interactive Scene Understanding

Figure 5: Informing LLM with embodied feedback enables many emergent capabilities, all of which are achieved without similar prompted examples. For instance, Inner Monologue can continually adapt to new instructions given by humans, propose new goals to achieve when faced with infeasibility for the previous plan, interact with humans in different natural languages, and answer questions about the current scene given past actions and feedback.

modes depending on the environment feedback. In-depth analysis on the failure causes indicates that *Success* and *Object* feedback can effectively reduce LLM planning failures and thus overall failure rate, albeit at the cost of introducing new failure modes to the system.

4.4 Emergent Capabilities

Although LLMs can generate fluent continuation from the prompted examples, we surprisingly find that, when informed with environment feedback, Inner Monologue demonstrates many impressive reasoning and replanning behaviors beyond the examples given in the prompt. Using a pre-trained LLM as the backbone, the method also inherits many of the appealing properties from its versatility and general-purpose language understanding. In this section, we demonstrate a few of these emergent capabilities.

Continued Adaptation to New Instructions. Although not explicitly prompted, the LLM planner can react to human interaction that changes the high-level goal mid-task. Fig 5a demonstrates a challenging case, where *Human* feedback changes the goal during the plan execution, and then changes the goal yet again by saying "finish the previous task". We can see that the planner incorporates the feedback correctly by switching tasks twice. In another instance, despite not being explicitly prompted to terminate after a human says "please stop", the LLM planner generalizes to this scenario and predicts a "done" action.

Self-Proposing Goals under Infeasibility. Instead of mindlessly following human-given instructions, Inner Monologue can also act as an interactive problem solver by proposing alternative goals to achieve when the previous goal becomes infeasible. In Fig 5b, to solve the task "put any two blocks inside the purple bowl", Inner Monologue first attempts an action of picking up the purple block – the action fails as the purple block is intentionally made to be too heavy for the robot. After a hint "the purple block is too heavy", it proposes to "find a lighter block" and successfully solves the task in the end.

Multilingual Interaction. Pre-trained LLMs are known to be able to translate from one language to another, without any finetuning. We observe that such multilingual understanding also transfers to the embodied settings considered in this work. Specifically, in Fig 5c, the human-provided new instruction is written in Chinese, but the LLM can correctly interpret it, re-narrate it as a concrete goal to execute in English, and accordingly replan its future actions. Occasionally, we find that this capability even extends to symbols and emojis.

Interactive Scene Understanding. We also observe that Inner Monologue demonstrates interactive understanding of the scene using the past actions and environment feedback as context. In Fig 5d, after a task instruction has been executed, we turn to ask questions about the scene, again a structure that has not appeared in the prompt. Surprisingly, we find that it can often correctly answer these questions that require temporal and embodied reasoning.

Robustness to Feedback Order. In the main experiments of the paper, we prompted the language model following certain conventions. For instance, in the simulated tabletop domain, the convention is `[Robot action, Scene, and Robot thought]`. In practice, we find that the LLM planner is robust to occasionally swapping the order of feedback. In Appendix Fig 9a, a new human instruction is injected in the middle of the plan execution, but this structure has not been seen in the example prompts. Yet the planner recognizes the change and generates a new "Robot thought: Goal state is..." statement allowing it to solve the new task.

Robustness to Typos. Inherited from the LLM backbone, our approach is robust to typos in human instruction, as seen in Appendix Fig 9b.

Despite the appealing findings about these emergent capabilities, we observe that they are of varying levels of consistency when no similar examples have been provided in the prompt, likely limited by the current capabilities of the language models. However, we believe that further investigations into these behaviors and addressing their limitations would each lead to exciting future directions.

5 Limitations

Limiting assumptions and failure modes. In Sec 4.1 and Sec 4.3, we assume access to oracle scene descriptors in the form of human observers or scripted systems to provide textual description back to the LLM planner. We study the viability of learned systems scene desription and object recognition in Appendix Table 5. As for failure modes, Inner Monologue may fail due to several sources of errors: (1) success detections, (2) LLM planning errors, and (3) control errors. False negative predictions from the success detector lead to additional retry attempts, while false positive predictions add adversarial partial observability to the environment. In some instances, we found that the LLM planners ignored the environment feedback and still proposed policy skills involving objects not present in the scene.

Limitations of results and future work. The performance of low-level control policies limits not only overall high-level instruction completion performance, but also limits the scope of tasks that the LLM is able to reason over: no matter how much the LLM reasoning improves, it can still be bottlenecked by what low-level control policies are able to achieve. Future work remains open on several fronts. First, with advances in image/video captioning and visual-question answering, a fully automated system of Inner Monologue can be implemented without a human in the loop as an oracle. Second, improvements can be made on how to aggregate potentially inaccurate sources of information, such as using text to describe the uncertainty of the feedback modules, or including additional feedback modules for safety and ethics for the proposed plans.

6 Conclusion

In this work, we investigated the role that environment feedback plays for LLMs reasoning in tasks involving embodied robotic planning and interaction. We presented a general formulation Inner Monologue that combines different sources of environment feedback with methods fusing LLM planning with robotic control policies. Then, we studied how these methods scaled to three different robotic manipulation settings in simulation and the real world. We found that environment feedback significantly improves high-level instruction completion, especially in challenging scenarios with adversarial disturbances. Finally, we analyze emergent capabilities of Inner Monologue that highlight how closed-loop language feedback enables replanning even in complex unseen settings.

Acknowledgments

The authors would like to thank Kanishka Rao and Vincent Vanhoucke for valuable feedback and discussions. In addition, the authors would like to acknowledge the large team who built [21], upon which we construct our Kitchen Mobile Manipulation experiments.

References

[1] L. P. Kaelbling and T. Lozano-Pérez. Integrated task and motion planning in belief space. *The International Journal of Robotics Research*, 32(9-10):1194–1227, 2013.

[2] A. G. Barto and S. Mahadevan. Recent advances in hierarchical reinforcement learning. *Discrete event dynamic systems*, 13(1):41–77, 2003.

[3] F. Petroni, T. Rocktäschel, P. Lewis, A. Bakhtin, Y. Wu, A. H. Miller, and S. Riedel. Language models as knowledge bases? *arXiv preprint arXiv:1909.01066*, 2019.

[4] Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know? *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

[5] J. Davison, J. Feldman, and A. M. Rush. Commonsense knowledge mining from pretrained models. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 1173–1178, 2019.

[6] A. Talmor, Y. Elazar, Y. Goldberg, and J. Berant. olmpics-on what language model pre-training captures. *Transactions of the Association for Computational Linguistics*, 8:743–758, 2020.

[7] A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

[8] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

[9] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. Chi, Q. Le, and D. Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

[11] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916*, 2022.

[12] A. K. Lampinen, I. Dasgupta, S. C. Chan, K. Matthewson, M. H. Tessler, A. Creswell, J. L. McClelland, J. X. Wang, and F. Hill. Can language models learn from explanations in context? *arXiv preprint arXiv:2204.02329*, 2022.

[13] M. Nye, A. J. Andreassen, G. Gur-Ari, H. Michalewski, J. Austin, D. Bieber, D. Dohan, A. Lewkowycz, M. Bosma, D. Luan, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.

[14] L. S. Vygotsky. *Thought and language*. MIT press, 2012.

[15] P. Carruthers. Thinking in language?: evolution and a modularist possibility. Cambridge University Press, 1998.

[16] L. Vygotsky. Tool and symbol in child development. *The vygotsky reader*, 1994.

[17] L. S. Vygotsky. Play and its role in the mental development of the child. *Soviet psychology*, 5(3): 6–18, 1967.

[18] C. Colas, T. Karch, C. Moulin-Frier, and P.-Y. Oudeyer. Vygotskian autotelic artificial intelligence: Language and culture internalization for human-like ai. *arXiv preprint arXiv:2206.01134*, 2022.

[19] A. Zeng, A. Wong, S. Welker, K. Choromanski, F. Tombari, A. Purohit, M. Ryoo, V. Sindhwani, J. Lee, V. Vanhoucke, et al. Socratic models: Composing zero-shot multimodal reasoning with language. *arXiv preprint arXiv:2204.00598*, 2022.

[20] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International Conference on Machine Learning*. PMLR, 2022.

[21] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, and M. Yan. Do as i can and not as i say: Grounding language in robotic affordances. In *arXiv preprint arXiv:2204.01691*, 2022.

[22] L. P. Kaelbling and T. Lozano-Pérez. Hierarchical planning in the now. In *Workshops at the Twenty-Fourth AAAI Conference on Artificial Intelligence*, 2010.

[23] S. Srivastava, E. Fang, L. Riano, R. Chitnis, S. Russell, and P. Abbeel. Combined task and motion planning through an extensible planner-independent interface layer. In *2014 IEEE international conference on robotics and automation (ICRA)*, 2014.

[24] R. E. Fikes and N. J. Nilsson. Strips: A new approach to the application of theorem proving to problem solving. *Artificial intelligence*, 1971.

[25] E. D. Sacerdoti. A structure for plans and behavior. Technical report, SRI International, Menlo Park California Artificial Intelligence Center, 1975.

[26] D. Nau, Y. Cao, A. Lotem, and H. Munoz-Avila. Shop: Simple hierarchical ordered planner. In *Proceedings of the 16th international joint conference on Artificial intelligence*, 1999.

[27] S. M. LaValle. *Planning algorithms*. Cambridge university press, 2006.

[28] M. Toussaint. Logic-geometric programming: An optimization-based approach to combined task and motion planning. In *Twenty-Fourth International Joint Conference on Artificial Intelligence*, 2015.

[29] M. A. Toussaint, K. R. Allen, K. A. Smith, and J. B. Tenenbaum. Differentiable physics and stable modes for tool-use and manipulation planning. *Robotics: Science and Systems Foundation*, 2018.

[30] B. Eysenbach, R. R. Salakhutdinov, and S. Levine. Search on the replay buffer: Bridging planning and reinforcement learning. *Advances in Neural Information Processing Systems*, 2019.

[31] D. Xu, S. Nair, Y. Zhu, J. Gao, A. Garg, L. Fei-Fei, and S. Savarese. Neural task programming: Learning to generalize across hierarchical tasks. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[32] D. Xu, R. Martín-Martín, D.-A. Huang, Y. Zhu, S. Savarese, and L. F. Fei-Fei. Regression planning networks. *Advances in Neural Information Processing Systems*, 32, 2019.

[33] T. Silver, R. Chitnis, N. Kumar, W. McClinton, T. Lozano-Perez, L. P. Kaelbling, and J. Tenenbaum. Inventing relational state and action abstractions for effective and efficient bilevel planning. *arXiv preprint arXiv:2203.09634*, 2022.

[34] D. Shah, P. Xu, Y. Lu, T. Xiao, A. Toshev, S. Levine, and B. Ichter. Value function spaces: Skill-centric state abstractions for long-horizon reasoning. *ICLR*, 2022. URL https://openreview.net/pdf?id=vgqS1vkkCbE.

[35] A. Srinivas, A. Jabri, P. Abbeel, S. Levine, and C. Finn. Universal planning networks: Learning generalizable representations for visuomotor control. In *International Conference on Machine Learning*, pages 4732–4741. PMLR, 2018.

[36] T. Kurutach, A. Tamar, G. Yang, S. J. Russell, and P. Abbeel. Learning plannable representations with causal infogan. *Advances in Neural Information Processing Systems*, 31, 2018.

[37] A. Akakzia, C. Colas, P.-Y. Oudeyer, M. Chetouani, and O. Sigaud. Grounding language to autonomously-acquired skills via goal generation. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=chPj_I5KMHG.

[38] S. Pirk, K. Hausman, A. Toshev, and M. Khansari. Modeling long-horizon tasks as sequential interaction landscapes. *arXiv preprint arXiv:2006.04843*, 2020.

[39] T. Kollar, S. Tellex, D. Roy, and N. Roy. Toward understanding natural language directions. In *2010 5th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 259–266. IEEE, 2010.

[40] S. Tellex, T. Kollar, S. Dickerson, M. Walter, A. Banerjee, S. Teller, and N. Roy. Understanding natural language commands for robotic navigation and mobile manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 25, pages 1507–1514, 2011.

[41] M. Bollini, S. Tellex, T. Thompson, N. Roy, and D. Rus. Interpreting and executing recipes with a cooking robot. In *Experimental Robotics*, pages 481–495. Springer, 2013.

[42] S. Tellex, R. Knepper, A. Li, D. Rus, and N. Roy. Asking for help using inverse semantics. 2014.

[43] T. Kollar, S. Tellex, D. Roy, and N. Roy. Grounding verbs of motion in natural language commands to robots. In *Experimental robotics*, pages 31–47. Springer, 2014.

[44] V. Blukis, Y. Terme, E. Niklasson, R. A. Knepper, and Y. Artzi. Learning to map natural language instructions to physical quadcopter control using simulated flight. *arXiv preprint arXiv:1910.09664*, 2019.

[45] S. Nair and C. Finn. Hierarchical foresight: Self-supervised learning of long-horizon tasks via visual subgoal generation. *ArXiv*, abs/1909.05829, 2020.

[46] F. Xia, C. Li, R. Martín-Martín, O. Litany, A. Toshev, and S. Savarese. Relmogen: Integrating motion generation in reinforcement learning for mobile manipulation. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[47] C. Li, F. Xia, R. Martin-Martin, and S. Savarese. Hrl4in: Hierarchical reinforcement learning for interactive navigation with mobile manipulators. In *Conference on Robot Learning*, 2020.

[48] Y. Jiang, S. Gu, K. Murphy, and C. Finn. Language as an abstraction for hierarchical deep reinforcement learning. In *NeurIPS*, 2019.

[49] D. Hafner, K.-H. Lee, I. Fischer, and P. Abbeel. Deep hierarchical planning from pixels. *arXiv preprint arXiv:2206.04114*, 2022.

[50] S. Mirchandani, S. Karamcheti, and D. Sadigh. Ella: Exploration through learned language abstraction. *Advances in Neural Information Processing Systems*, 34:29529–29540, 2021.

[51] P. A. Jansen. Visually-grounded planning without vision: Language models infer detailed plans from high-level instructions. *arXiv preprint arXiv:2009.14259*, 2020.

[52] P. Sharma, A. Torralba, and J. Andreas. Skill induction and planning with latent language. *arXiv preprint arXiv:2110.01517*, 2021.

[53] S. Li, X. Puig, Y. Du, C. Wang, E. Akyurek, A. Torralba, J. Andreas, and I. Mordatch. Pre-trained language models for interactive decision-making. *arXiv preprint arXiv:2202.01771*, 2022.

[54] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[55] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[56] N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.

[57] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

[58] C. Paxton, Y. Bisk, J. Thomason, A. Byravan, and D. Foxl. Prospection: Interpretable plans from language by predicting the future. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 6942–6948. IEEE, 2019.

[59] S. Stepputtis, J. Campbell, M. Phielipp, S. Lee, C. Baral, and H. Ben Amor. Language-conditioned imitation learning for robot manipulation tasks. *Advances in Neural Information Processing Systems*, 33:13139–13150, 2020.

[60] V. Blukis, R. A. Knepper, and Y. Artzi. Few-shot object grounding and mapping for natural language robot instruction following. *arXiv preprint arXiv:2011.07384*, 2020.

[61] C. Lynch and P. Sermanet. Language conditioned imitation learning over unstructured data. *Robotics: Science and Systems*, 2021. URL https://arxiv.org/abs/2005.07648.

[62] Y. Chen, R. Xu, Y. Lin, and P. A. Vela. A joint network for grasp detection conditioned on natural language commands. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4576–4582. IEEE, 2021.

[63] O. Mees, L. Hermann, and W. Burgard. What matters in language conditioned robotic imitation learning. *arXiv preprint arXiv:2204.06252*, 2022.

[64] C. Yan, F. Carnevale, P. Georgiev, A. Santoro, A. Guy, A. Muldal, C.-C. Hung, J. Abramson, T. Lillicrap, and G. Wayne. Intra-agent speech permits zero-shot task acquisition. *arXiv preprint arXiv:2206.03139*, 2022.

[65] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pages 8748–8763. PMLR, 2021.

[66] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[67] J. Lu, D. Batra, D. Parikh, and S. Lee. Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks. *Advances in neural information processing systems*, 32, 2019.

[68] Z. Wang, J. Yu, A. W. Yu, Z. Dai, Y. Tsvetkov, and Y. Cao. Simvlm: Simple visual language model pretraining with weak supervision. *arXiv preprint arXiv:2108.10904*, 2021.

[69] A. Suglia, Q. Gao, J. Thomason, G. Thattai, and G. Sukhatme. Embodied bert: A transformer model for embodied, language-guided visual task completion. *arXiv preprint arXiv:2108.04927*, 2021.

[70] T. Chen, S. Kornblith, K. Swersky, M. Norouzi, and G. E. Hinton. Big self-supervised models are strong semi-supervised learners. *Advances in neural information processing systems*, 33:22243–22255, 2020.

[71] A. Jain, M. Guo, K. Srinivasan, T. Chen, S. Kudugunta, C. Jia, Y. Yang, and J. Baldridge. Mural: multimodal, multitask retrieval across languages. *arXiv preprint arXiv:2109.05125*, 2021.

[72] J. Sun, D.-A. Huang, B. Lu, Y.-H. Liu, B. Zhou, and A. Garg. Plate: Visually-grounded planning with transformers in procedural tasks. *IEEE Robotics and Automation Letters*, 7(2):4924–4930, 2022.

[73] F. Sener and A. Yao. Zero-shot anticipation for instructional activities. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 862–871, 2019.

[74] A. Khandelwal, L. Weihs, R. Mottaghi, and A. Kembhavi. Simple but effective: Clip embeddings for embodied ai. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14829–14838, 2022.

[75] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee. Transporter networks: Rearranging the visual world for robotic manipulation. *Conference on Robot Learning (CoRL)*, 2020.

[76] M. Shridhar, L. Manuelli, and D. Fox. Cliport: What and where pathways for robotic manipulation. In *Conference on Robot Learning*, pages 894–906. PMLR, 2022.

[77] X. Gu, T.-Y. Lin, W. Kuo, and Y. Cui. Open-vocabulary object detection via vision and language knowledge distillation. *arXiv preprint arXiv:2104.13921*, 2021.

[78] I. Lenz, H. Lee, and A. Saxena. Deep learning for detecting robotic grasps. *The International Journal of Robotics Research*, 34(4-5):705–724, 2015.

[79] F.-J. Chu, R. Xu, and P. A. Vela. Real-world multiobject, multigrasp detection. *IEEE Robotics and Automation Letters*, 3(4):3355–3362, 2018.

[80] D. Kalashnikov, J. Varley, Y. Chebotar, B. Swanson, R. Jonschkowski, C. Finn, S. Levine, and K. Hausman. Mt-opt: Continuous multi-task robotic reinforcement learning at scale. *arXiv preprint arXiv:2104.08212*, 2021.

[81] T. Migimatsu and J. Bohg. Grounding predicates through actions. *arXiv preprint arXiv:2109.14718*, 2021.

[82] Y. Cui, S. Niekum, A. Gupta, V. Kumar, and A. Rajeswaran. Can foundation models perform zero-shot task specification for robot manipulation? In *Learning for Dynamics and Control Conference*, pages 893–905. PMLR, 2022.

[83] M. Liang and X. Hu. Recurrent convolutional neural network for object recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3367–3375, 2015.

[84] S. Ren, K. He, R. Girshick, and J. Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, 28, 2015.

[85] Z. Zou, Z. Shi, Y. Guo, and J. Ye. Object detection in 20 years: A survey. *arXiv preprint arXiv:1905.05055*, 2019.

[86] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[87] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh. Vqa: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 2425–2433, 2015.

[88] L. Zhou, H. Palangi, L. Zhang, H. Hu, J. Corso, and J. Gao. Unified vision-language pre-training for image captioning and vqa. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 13041–13049, 2020.

[89] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han. Once for all: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations*, 2020. URL https://arxiv.org/pdf/1908.09791.pdf.

[90] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, et al. Flamingo: a visual language model for few-shot learning. *arXiv preprint arXiv:2204.14198*, 2022.

[91] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[92] A. Kamath, M. Singh, Y. LeCun, G. Synnaeve, I. Misra, and N. Carion. Mdetr-modulated detection for end-to-end multi-modal understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1780–1790, 2021.

[93] T. Xiao, E. Jang, D. Kalashnikov, S. Levine, J. Ibarz, K. Hausman, and A. Herzog. Thinking while moving: Deep reinforcement learning with concurrent control. *arXiv preprint arXiv:2004.06089*, 2020.

A Inner Monologue Implementation Details

We study three different implementations of Inner Monologue for each of the experimental settings. While each version incorporates textual environment feedback into planning, there are differences in the internal components of each system, as seen in Table 4.

	Tabletop Rearrangement (Sim)	**Tabletop Rearrangement (Real)**	**Kitchen Mobile Manipulation (Real)**
LLM	InstructGPT-1.3B	InstructGPT-1.3B	PALM-540B
Policies	CLIPort + TransporterNets	MDETR + heuristics	Learned policies
***Success* Feedback**	Ground truth + heuristics	MDETR + heuristics	Finetuned CLIP
***Object* Feedback**	Ground truth + heuristics	MDETR	Human-provided (structured)
***Scene* Feedback**	Ground truth + heuristics	N/A	N/A
***Human* Feedback**	N/A	N/A	Human-provided (unstructured)

Table 4: Comparison between different versions of Inner Monologue implemented in three different environments.

A.1 Inner Monologue for Simulated Tabletop Rearrangement

Large Language Model We use InstructGPT [91], a 1.3B parameter language model fine-tuned from GPT-3 [9] with human feedback, accessed through OpenAI API.

Low-level Policies We use a CLIP-based [65] Transporter Net [75] (inspired by CLIPort [76]) as the pick-and-place primitive that can be invoked by the LLM planner. The policy is trained on 20000 pre-collected demonstrations, where each demonstration contains 1) language instruction of the format "pick up [x] and place it on [y]", 2) top-down view of RGB-D observation of the current environment, 3) expert pick coordinates, and 4) expert place coordinates. The expert pick and place coordinates are obtained by accessing ground-truth object pose in the simulator. Unlike the evaluated settings in [76], the demonstrations cover all objects because the focus of this work is not on generalization across object instances but on novel long-horizon behaviors. The policy outputs a 3D pick location and a 3D place location, and a scripted pick-and-place motion is executed following the predicted locations.

Environment Feedback: Object Recognition We provide the list of objects present in the scene at the start of each episode for the language model (without bounding boxes or coordinates because spatial manipulation is handled by low-level policies). Although it has been shown in [19] that this can be obtained by off-the-shelf pre-trained open-vocabulary object detection models such as ViLD [77] and MDETR [92], we obtain the list of objects by accessing simulator state for simplicity.

Environment Feedback: Success Detection For *Object* + *Success* method, we provide textual feedback of low-level policy success detection results after each policy execution. The success detector is built using heuristics based on ground-truth object poses: a pick-and-place execution is successful if picked object has a 2D Euclidean distance $<4\,\mathrm{cm}$ from the place object and its height is greater than the place object. If the place object is a location (e.g., top side of the table) rather than a block or a bowl, the height requirement is not enforced.

Environment Feedback: Passive Scene Description For *Object* + *Scene* method, we provide task-progress scene description as a list of achieved sub-goals after each pick-and-place execution. At the start of the action plan, the language model first generates a list of desired sub-goals given the high-level instruction. The sub-goals are of the format "[x] is on top of [y]". After each pick-and-place execution, we iterate through all desired sub-goals inferred by the language model, and check which are satisfied using the success detector described above. The full list of currently satisfied sub-goals are appended to the language model prompt. Additionally, we allow the language model to generate chain-of-thought summarization following the achieved sub-goals (i.e., "Robot thought: ..."), which we find to be useful empirically.

A.2 Inner Monologue for Real-World Tabletop Rearrangement

Large Language Model We use InstructGPT [91], a 1.3B parameter language model fine-tuned from GPT-3 [9] with human feedback, accessed through OpenAI API.

Low-level Policies We use a single low-level policy for the real tabletop rearrangement environment that is responsible for performing object-centric pick and place actions as instructed by the language model. The policy takes as input 1) the bounding boxes of all the objects in the scene, 2) the names of the object to pick and the target object for place, 3) the depth image corresponding to the RGB image from which object detection was done, 4) camera intrinsics, and 5) camera pose. It then outputs a 3D pick position and a 3D place location for the robot to follow.

The pick position is the 3D coordinate of the center of the object bounding box in the robot base frame. This is obtained by first deprojecting the depth image pixel corresponding to the center of the pick object bounding box with the given camera intrinsics, then transforming this point from the camera frame to the robot frame with the given camera pose. Camera pose is known because it is a wrist camera, and we register the delta pose from the mounted camera to the robot end-effector.

For the block stacking task, the place location is obtained the same way as the pick position. For the object sorting task, the place location is chosen to be a point in the target plate's bounding box that is farthest way from the bounding boxes of other objects. This is done to prevent undesired stacking behavior when placing objects into the plate, which may cause the object to roll off and fall off the table.

To perform the pick and place motions, the robot moves to a position $15\,\mathrm{cm}$ above the intended pick or place position, and then it slowly lowers the end-effector until a $5\,\mathrm{N}$ contact force is detected. This is more robust than moving directly to the height obtained from the depth image due to depth sensor noises. We set a threshold for how low the end-effector can go for the object sorting task so picking inaccuracies do not result in accidentally picking up the plate.

We add planar translation noise to the pick position by sampling the magnitude from a zero-mean Normal distribution and then sampling an angle from a uniform distribution on the unit circle. The standard deviation for the Normal distribution is $\sigma = 1.5\,\mathrm{cm}$ for the block stacking task, $\sigma = 0.7\,\mathrm{cm}$ for the object sorting task, and the samples are capped at 1.5σ.

Object Recognition Implementation Object detection is done by MDETR [92], an open-vocabulary object detection model. It takes as input an RGB image with an image caption, and the model tries to identify the objects described in the caption. To form the caption for our tasks, we join the list of all possible object names together, separated by periods. Note that not all provided objects will be detected due to occlusions. MDETR detections are not always robust or consistent. For example, the detected object bounding box may some times be bigger or smaller than the object, or its center may be far away from the object center. Its performance also drops when objects are in close clutter. These issues with perception further motivate the need for success detection and multi-step environment feedback, so the high-level language model planner can be robust to these inaccuracies.

Environment Feedback: Success Detection In the real tabletop rearrangement environment, success detection is done by comparing the 2D position of the center of the detected object bounding box after pick and place action with the intended 2D place position. By 2D we mean the x-y spatial coordinates in the base frame of the robot, not the image coordinates. These are obtained by deprojecting the detected bounding box centers from the depth image and converting the resulting 3D point from camera frame to base frame. The detector returns success if the difference between the two positions is less than a threshold. For the block stacking task, the threshold is $3\,\mathrm{cm}$, and for the object sorting task it is $10\,\mathrm{cm}$. While $3\,\mathrm{cm}$ is larger than half of the dimension of the blocks we use ($4\,\mathrm{cm}$), it is more robust for our case because the center of the detected object bounding box is often not the center of the block. The threshold for object sorting is much higher, because for some objects, once they are placed in the plate, they may roll around until contact with the plate boundary.

Environment Feedback: Object Recognition For the block stacking task, the scene description contains a list of currently visible objects and a list of previously visible objects that are no longer visible. We may add new objects to the currently visible objects list if, after a robot action, MDETR detects an object that we have not seen before. This happens for the block stacking task as there is an initial partially constructed tower of two blocks, the bottom of which is initially occluded.

For the object sorting task, the scene description contains a list of currently visible objects and a list of objects that the robot has successfully moved into a plate. Like the block stacking task, some objects are initially occluded, so later actions may reveal them and they will be added to the visible objects list. The successfully moved object list is needed so that the planner does not stop before task completion and that it stops on task completion.

A.3 Inner Monologue for Real-World Mobile Manipulation in a Kitchen Setting

Large Language Model We use PALM [8], a 540B parameter language model trained on a large datasets that include high-quality web documents, books, Wikipedia, conversations, and GitHub code.

Low-level Policies Following the implementation described in SayCan [21], we use a combination of learned and scripted control policies for navigation and manipulation. The learned manipulation policies responsible for counter picking, drawer opening and closing, drawer picking, and countertop object manipulation are Behavior Cloning (BC) policies trained on 68000 teleoperated demonstrations and 12000 autonomous successes that were collected over the course of 11 months using a fleet of 10 robots. The teleoperators use VR headset controllers to track the motion of their hand, which is then mapped onto the robot's end-effector pose. The scripted navigation policies utilize a ground-truth map along with a learned perception module to navigate between different points in the environment. The scripted manipulation policy is solely responsible for countertop placing when preceded by a navigation policy, and follows pre-computed motions. The Value Functions used by SayCan for affordance grounding are provided by the Q-networks of trained RL agents; we follow the RL training setup described in [21].

Environment Feedback: Object Recognition We use human-provided object recognition to provide feedback about the presence of objects visible to the robot camera. For example, if there were only a coke can and an apple on top of the kitchen counter, then the human-provided object recognition feedback would appear as "`[scene: coke can, apple]`".

While we utilize human-provided object recognition to gauge the role of maximally accurate *Object* feedback, we also study the feasibility of using learned object recognition models to provide fully automated *Object* feedback. We find that two such models, ViLD [77] and MDETR [92], perform worse than humans but still quite resonably at providing *Object* feedback, even without domain-specific data for fine-tuning. As advances in computer vision improve object detection models that can transfer zero-shot or few-shot to novel environments like our kitchen environment, we expect that fully automated *Object* feedback will increasingly approach human-provided accuracy.

	ViLD	MDETR
Precision	85.7%	39.6%
Recall	72.0%	87.5%
Accuracy	88.9%	68.2%

Table 5: Comparison of ViLD [77] and MDETR [92], two open-vocabulary object detection models. We evaluate these two pretrained models on 10 representative kitchen mobile manipulation episodes. ViLD has strong overall accuracy, but still fails to detect objects 28.0% of the time.

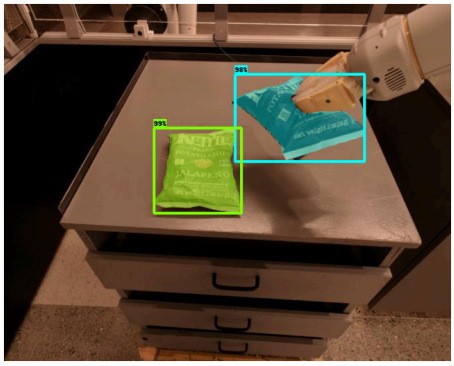


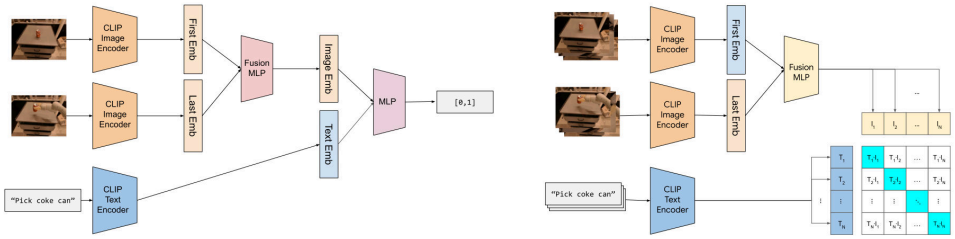
Figure 6: An example of ViLD object detection segmentation mask and bounding box predictions.

Environment Feedback: Success Detection We trained a "foresight" success detector on offline data collected from a combination of teleoperated demonstrations and autonomous roll-outs performing the low-level tasks. The input to the model consists of: (1) o_0, the initial image observation, (2) o_f, the final image observation after the policy chose to terminate its chosen skill execution, and (3) l_k, the low-level skill which the agent was trying to execute (e.g., "`Pick coke can`"). The model uses image encoders from CLIP [65] to embed o_0 and o_f, concatenate them, and fuse these representations with a fusion MLP. This image embedding is concatenated with the text embedding obtained by the CLIP text encoder, then passed through another MLP. The output of the model is a scalar denoting the probability of the agent succeeding at the specified task (Fig 7a). The model is trained with the binary cross entropy loss with respect to the ground truth binary label. We fine tune the CLIP model as well during the training. At inference time within Inner Monologue, we output the text "`[success: no]`" when the probability is below a certain threshold.

To reduce false positive predictions, we trained a separate "hindsight" success predictor model. Given the first and last observation, the model outputs a probability distribution over all the possible skills. We used this model to disambiguate between possible tasks that the agent may have achieved. To train this model, we use the symmetric contrastive loss as used in CLIP (Fig 7b). At inference time, similar to the CLIP

model, we take a dot product between the image-fusion embedding and the text embeddings and apply softmax with a learned temperature.

We combine the foresight and hindsight model by first thresholding the probability from the foresight model by some value τ. If the probability is above, then we run the hindsight model and only predict success if the argmax across all skills is indeed the foresight task.



(a) Foresight success detector

(b) Hindsight success detector

Figure 7: Success Detection architecture used for the Kitchen Environment. (*Left*) The foresight model predicts whether a given instruction was successfully achieved between the first and last image. (*Right*) The hindsight model fine-tuned via contrastive loss as used in CLIP [65]. At inference time, the model is used infer among the possible instructions which one was achieved.

Environment Feedback: Active Scene Description We perform a case study where we allow the LLM agent to ask questions and source *Human* feedback directly. After each LLM planning step, the LLM chooses between `and continue` or `and ask`: for the `and continue` option the LLM proceeds to the planning the next action to execute, and for the `and ask` option the LLM proceeds to propose a question to ask the human. The questions proposed by the LLM do not follow any pre-determined format or scope, so human-provided answers are similarly open-ended. These answers can range from confirming factual observations about the scene to providing refinements or changes to the high-level plan. Some examples can be found in Sec D.

B Experiment Details

B.1 Simulated Tabletop Rearrangement Environment

There are a total of 8 tasks, listed in Table 1. For all tasks, up to 4 blocks and up to 3 bowls are randomly initialized with different colors (chosen from 10 different colors) and initial locations. A 15 cm distance is enforced between any two objects to avoid collision and penetration at initialization. For tasks that involve a specific object instance or location in the task instruction (e.g., "Put all the blocks in the [x] bowl"), a random task instruction is sampled using the available objects in the scene or all available locations. There are a total of a total of 9 locations (e.g., "top side", "bottom right corner", etc). To allow for automatic evaluations, a binary reward function is defined for each task using ground-truth state of the objects.

CLIPort Baseline We additionally train a multi-task CLIPort [76] policy on 4 of the total 8 tasks as an additional baseline (see Table 1 for train/test split). The goal is to evaluate whether solitary language-conditioned policies can perform well on long-horizon tasks and generalize to new task instructions. Note that this is a different policy as the low-level primitive used in Inner Monologue, though sharing the same architecture. The policy also shares the same training details as the low-level policy used in Inner Monologue but is trained on 20000 demonstrations across 4 training tasks.

B.2 Real Tabletop Rearrangement

There are two tasks in the real tabletop rearrangement domain – a block stacking task and an object sorting task.

In the block stacking task, the initial state consists of three 4 cm cubes, two of which have been already stacked. The robot needs to complete stacking the full tower either by putting the third block onto the partially stacked two, or by putting the two partially stacked blocks onto the third block. Due to occlusions, the robot only perceives two blocks in the beginning, so without scene description feedback, the planner has a 50% chance of completing the task.

For the object sorting task, the initial state consists of three fruits, three bottles, and three plates. The fruits and bottles are placed in a clutter in one of the three plates, and the robot is asked to sort them by putting fruits on one of the remaining plates, and bottles on the other. Due to occlusions and reduced object detection performance in clutter, some objects may not be detected in the initial clutter, so scene description feedback is still needed. The three bottles used are a ketchup bottle, a mustard bottle, and a soft drink bottle. The three fruits used are an apple, a strawberry, and a plum. Note these are toy objects made out of plastic, so they are smaller and lighter than their real-world counterparts, and they are also rigid, allowing for robust suction grasps. They have realistic visual appearances, which allow MDETR detections.

B.3 Real Kitchen Mobile Manipulation

We follow the environment and open-ended task setup described in SayCan [21]. An Everyday Robots robot, a mobile manipulator robot with RGB observations, is placed in a kitchen with multiple named locations that contain a set of 15 household items. Given an unstructured human instruction, the robot must complete the instruction successfully by executing a set of navigation and manipulation skills, which are vision-based concurrent [93] continuous closed-loop policies. Fig 8 presents a high-level overview of the environment. The full list of tasks are shown in Table 6.

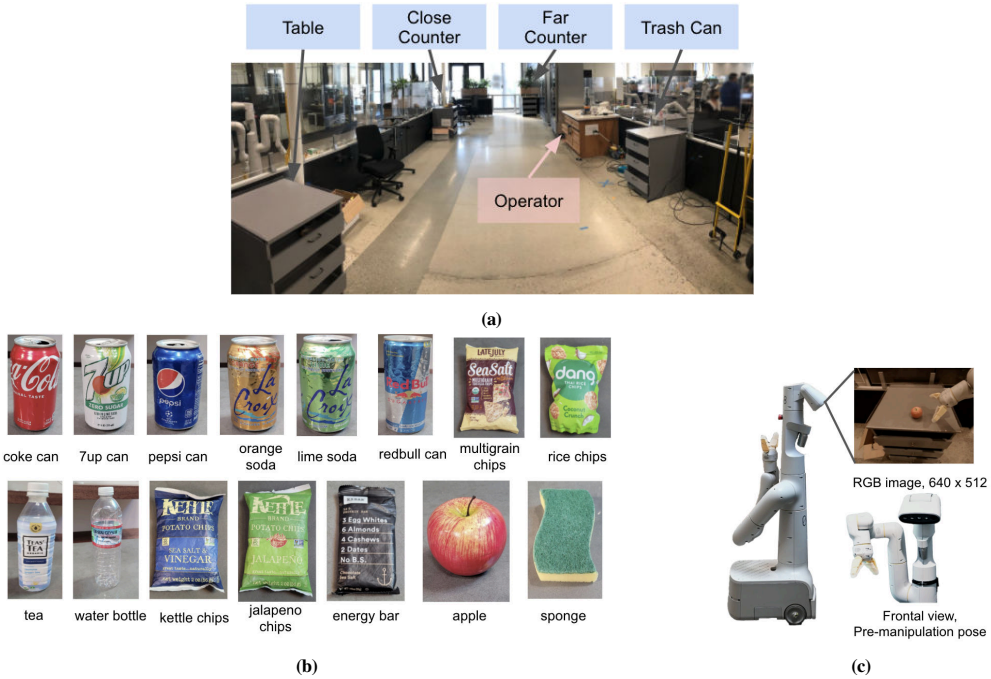


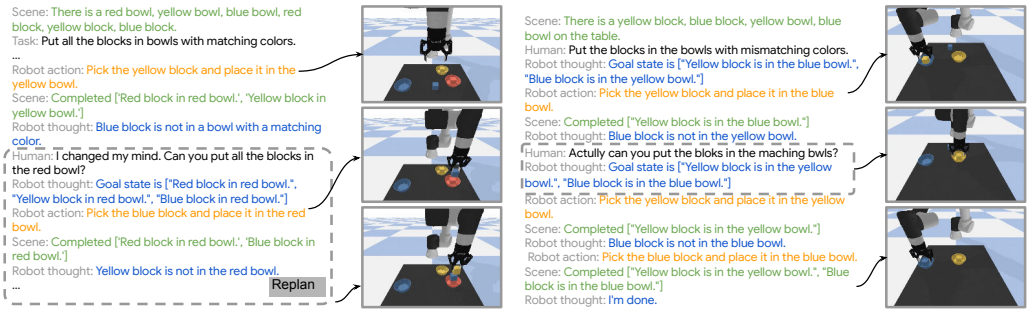
Figure 8: The experiments were performed in a mock office kitchen mirroring this setup (a), with 5 predicted locations and 15 objects (b). The robot is a mobile manipulator with a 7 degree-of-freedom arm and a gripper with policies trained from an RGB observation (c).

	Task Instructions
Manipulation	"pick up a soda" "pick up the jalapeno or kettle chips" "pick up a snack" "pick up a caffeinated drink"
Mobile Manipulation	"can you throw away the soda on the table" "I spilled my coke, can you bring me something to clean it up"
Drawer Manipulation	"leave the top drawer open" "put a coke in the top drawer"

Table 6: List of all instructions in Kitchen Mobile Manipulation tasks. We evaluate the algorithm on 3 families of instructions on a mock kitchen scene.

C Additional Results

We show additional results of emergent capability in Fig 9. Please refer to Sec 4.4 for context.



(a) Robustness to Feedback Order

(b) Robustness to Typos

Figure 9: Additional emergent capabilities in Inner Monologue. While we mostly use a fixed feedback order for each domain, we find that Inner Monologue can seamlessly handle different feedback injected into the prompt, as opposed to the normal case of feedback being provided after the prompt and after planning steps (*left*). Additionally, it is also robust to typos in the instruction, an ability likely inherited from the backbone LLM (*right*).

D Prompts

Listing 1: Prompt in Inner Monologue on Simulated Tabletop Rearrangement

```
==============
Scene: There is a cyan, yellow, brown block.
Human: Move all the blocks to the top left corner.
Robot thought: Goal state is ["Cyan block is
       on the top left corner.", "Yellow block is on the top left corner.", "Brown block is on the top left corner."]

Robot action: Pick the cyan block and place it on the top left corner.
Scene: You have completed ["Cyan block is on the top left corner."]
Robot thought: Yellow and brown blocks are not on the top left corner.

Robot action: Pick the yellow block and place it on the top left corner.
Scene: You have completed ["Cyan block is on the top left corner."]
Robot thought: Yellow and brown blocks are not on the top left corner.

Robot action: Pick the yellow block and place it on the top left corner.
Scene: You have completed ["Cyan block is on the top left corner.", "Yellow block is on the top left corner."]
Robot thought: Brown block is not on the top left corner."

Robot action: Pick the brown block and place it on the top left corner.
Scene: You have completed ["Yellow block is on the top left corner.", "Brown block is on the top left corner."]
Robot thought: Cyan block is not on the top left corner

Robot action: Pick the cyan block and place it on the top left corner.
Scene: You have completed ["Yellow block
       is on the top left corner.", "Brown block is on the top left corner.", "Cyan block is on the top left corner."]
Robot thought: None.

==============
Scene: There is a pink, gray, orange block.
Human: Move all the blocks to the bottom side.
Robot thought: Goal state is ["
      Pink block is on the bottom side.", "Gray block is on the bottom side.", "Orange block is on the bottom side."].

Robot action: Pick the pink block and place it on the bottom side.
Scene: You have completed ["Pink block is on the bottom side."]
Robot thought: Gray and orange blocks are not on the bottom side.

Robot action: Pick the gray block and place it on the bottom side.
Scene: You have completed ["Pink block is on the bottom side.", "Gray block is on the bottom side."]
Robot thought: Orange block is not on the bottom side.

Robot action: Pick the orange block and place it on the bottom side.
Scene: You have completed ["Orange block is on the bottom side."]
Robot thought: Pink and gray blocks are not on the bottom side.

Robot action: Pick the gray block and place it on the bottom side.
Scene: You have completed ["Pink block is on the bottom side.", "Gray block is on the bottom side."]
Robot thought: Orange block is not on the bottom side.

Robot action: Pick the orange block and place it on the bottom side.
Scene: You have completed ["
      Pink block is on the bottom side.", "Gray block is on the bottom side.", "Orange block is on the bottom side."]
Robot thought: None.
```

Listing 2: Prompt in Inner Monologue on Real Tabletop Rearrangement

```
==============
Task: Move all blocks to the top left corner

Scene: Visible objects are ["cyan block", "yellow block", "brown block"]
Scene: Occluded objects are []
Robot action: robot.pick_place("cyan block", "top left corner")
Successful action: True

Scene: Visible objects are ["cyan block", "yellow block", "brown block"]
Scene: Occluded objects are []
Robot action: robot.pick_place("yellow block", "top left corner")
Successful action: False

Scene: Visible objects are ["cyan block", "yellow block", "brown block"]
Scene: Occluded objects are []
Robot action: robot.pick_place("yellow block", "top left corner")
Successful action: True

Scene: Visible objects are ["yellow block", "brown block"]
Scene: Occluded objects are ["cyan block"]
Robot action: robot.pick_place("brown block", "top left corner")
Successful action: True
```

Scene: Visible objects are ["brown block"]
Scene: Occluded objects are ["cyan block", "yellow block"]
Robot action: robot.stop()
STOP

==========
Task: Stack all the blocks.

Scene: Visible objects are ["brown block", "purple block"]
Scene: Occluded objects are []
Robot action: robot.pick_place("brown block", "purple block")
Successful action: False

Scene: Visible objects are ["brown block", "purple block"]
Scene: Occluded objects are []
Robot action: robot.pick_place("brown block", "purple block")
Successful action: True

Scene: Visible objects are ["brown block", "orange block"]
Scene: Occluded objects are ["purple block"]
Robot action: robot.pick_place("orange block", "brown block")
Successful action: True

Scene: Visible objects are ["orange block"]
Scene: Occluded objects are ["purple block", "brown block"]
Robot action: robot.stop()
STOP

==========
Task: Stack all the blocks.

Scene: Visible objects are ["brown block", "purple block"]
Scene: Occluded objects are []
Robot action: robot.pick_place("brown block", "purple block")
Successful action: True

Scene: Visible objects are ["brown block"]
Scene: Occluded objects are ["purple block"]
Robot action: robot.stop()
STOP

==========
Task: Put objects in their corresponding bowls

Scene: Visible objects are ["red block", "green block", "red bowl", "green bowl"]
Scene: Occluded objects are []
Robot action: robot.pick_place("red block", "red bowl")
Successful action: False

Scene: Visible objects are ["red block", "green block", "red bowl", "green bowl"]
Scene: Occluded objects are []
Robot action: robot.pick_place("red block", "red bowl")
Successful action: True

Scene: Visible objects are ["red block", "green block", "red bowl", "green bowl"]
Scene: Occluded objects are []
Robot action: robot.pick_place("green block", "green bowl")
Successful action: True

Scene: Visible objects are ["red block", "green block", "red bowl", "green bowl"]
Scene: Occluded objects are []
Robot action: robot.stop()
STOP

==========
Task: Sort pencils and markers and in different containers

Scene: Visible objects are ["red marker", "red pen", "blue box", "green bowl"]
Scene: Occluded objects are []
Robot action: robot.pick_place("red marker", "green bowl")
Successful action: False

Scene: Visible objects are ["red marker", "red pen", "blue box", "green bowl"]
Scene: Occluded objects are []
Robot action: robot.pick_place("red marker", "green bowl")
Successful action: False

Scene: Visible objects are ["red marker", "red pen", "blue box", "green bowl"]
Scene: Occluded objects are []
Robot action: robot.pick_place("red marker", "green bowl")
Successful action: True

Scene: Visible objects are ["red marker", "red pen", "blue box", "green bowl"]
Scene: Occluded objects are []
Robot action: robot.pick_place("red pen", "blue box")

```
Successful action: True

Scene: Visible objects are ["red marker", "red pen", "blue box", "green bowl"]
Scene: Occluded objects are []
Robot action: robot.stop()
STOP

===============
```

Listing 3: Prompt in Inner Monologue in Kitchen mobile manipulation environment.

```
Robot: Hi there, I'm a robot operating in an office kitchen.
You can ask me to do various tasks and I'll tell you the sequence of actions I would do to accomplish your task.
Human: hold the snickers
Robot: 1. pick up the snickers[success: no][scene: snickers]
2. pick up the snickers
2. done.
Human: put the trailmix here
Robot: 1. put down the trailmix
2. done.
Human: put a water bottle and an oatmeal next to the microwave
Robot: 1. find a water
2. pick up the water[success: no]
3. pick up the water[success: no]
4. pick up the water
5. go to microwave
6. put down the water[scene: water, microwave]
7. find an oatmeal
8. pick up the oatmeal[scene: oatmeal]
9. go to the microwave
10. put down the oatmeal[scene: microwave, oatmeal, water]
11. done.
Human: put a grapefruit from the table into the bowl
Robot: 1. find a grapefruit
2. pick up the grapefruit
3. go to the bowl
4. put down the grapefruit
5. done.
Human: get a sponge from the counter and put it in the sink
Robot: 1. find a sponge
2. pick up the sponge[success: no][scene: sponge]
3. pick up the sponge[success: no]
4. pick up the sponge[scene: sponge]
5. go to the sink
6. put down the sponge
7. done.
Human: move a drink from the table to the counter
Robot: 1. find a water
2. pick up the water[scene: tea, grapefruit soda, kettle chips]
3. pick up the grapefruit soda
4. go to the counter
5. put down the grapefruit soda[success: no]
6. put down the grapefruit soda
7. done.
Human: bring me some snacks
Robot: 1. find a twix
2. pick up the twix[scene: pretzel]
3. find a pretzel
4. pick up the pretzel
5. bring it to you
6. put down the pretzel[scene: pretzel, dried fruit]
7. find an dried fruit
8. pick up the dried fruit
9. bring it to you
10. put down the dried fruit[scene: ]
11. done.
Human: bring me a fruit
Robot: 1. find an orange
2. pick up the orange[scene: apple, redbull]
3. pick up the apple[success: no]
5. pick up the apple
6. bring it to you
7. put down the apple
8. done.
Human: deliver a can of coke to me
Robot: 1. find a coke
2. pick up the coke
3. bring it to you
4. put down the coke
5. done.
Human: I finished my drink, can you throw it away and bring me a mountain dew
Robot: 1. find a sprite
2. pick up the sprite[scene: coke]
3. pick up the coke[success: no][scene: coke]
```

```
4. pick up the coke[scene: coke]
5. go to the trash
6. put down the coke
7. find a mountain dew
8. pick up the mountain dew
9. bring it to you
10. put down the mountain dew
11. done.
Human: let go of the banana
Robot: 1. put down the banana
2. done.
Human: put the grapes in the bowl and then move the cheese to the table
Robot: 1. find the grapes
2. pick up the grapes
3. go to the bowl
4. put down the grapes
5. pick up the cheese[success: no]
6. pick up the cheese
7. go to the table
8. put down the cheese
9. tell you I'm done.
Human: pick up the peanuts
Robot: 1. pick up the peanuts
2. done.
Human: bring me some snack
Robot: 1. pick up the peanuts
2. pick up the peanuts[scene: blue chip bag, 7up]
3. pick up the blue chip bag
4. bring it to you
5. put down the blue chip bag
5. done.
Human: throw away a coffee cup
Robot: 1. find a coffee cup
2. pick up the coffee cup[success: no]
3. pick up the coffee cup
4. go to the trash
5. put down the coffee cup
6. done.
Human: place a knife and a banana to the table
Robot: 1. find a knife
2. pick up the knife
3. pick up the knife
4. go to the table
5. put down the knife
6. find a banana
7. pick up the banana
8. go to the table
9. put down the banana[scene: banana, knife]
10. done.
```

Listing 4: Prompt in Inner Monologue in Kitchen mobile manipulation environment with Human interaction

```
Robot: Hi there, I'm a robot operating in an office kitchen.
You can ask me to do various tasks and I'll tell you the sequence of actions I would do to accomplish your task.
Human: hold the snickers
Robot: 1. pick up the snickers and ask: Because snickers may
       be too thin to be picked up by my gripper, was I successful at picking it up? Human: Yes, you were successful
2. done.
Human: put the trailmix here
Robot: 1. put down the trailmix and continue
2. done.
Human: can you bring me a drink from the counter?
Robot: 1. go to counter and ask: Do you have a preference on what
      drink you would want? Human: I don't want to take in too much sugar, so please bring me anything that's healthier
2. pick up tea and continue
3. bring it to you and continue
4. put down tea, Human: Actually, can you also bring me an energy bar?
5. find energy bar and continue
6. pick up energy bar and continue
7. bring it to you and continue
8. put down energy bar and continue
9. done.
Human: get a sponge from the counter and put it in the sink
Robot: 1. find a sponge and continue
2. pick up the sponge and continue
3. pick up the sponge and continue
4. pick up the sponge and continue
5. go to the sink and continue
6. put down the sponge and continue
7. done.
Human: take any food item from the table and put them next to the microwave
Robot: 1. go
       to table and ask: What food items are there on the table? Human: There are kettle chips and apple on the table
2. find kettle chips and ask: Is this what you want? Human: No, please get me apple instead
```

3. find an apple and continue
4. pick up the apple and continue
5. go to the microwave and continue
6. put down the apple and continue
7. done.
Human: put the grapes in the bowl and then move the cheese to the table
Robot: 1. find the grapes and continue
2. pick up the grapes and continue
3. go to the bowl and continue
4. put down the grapes and continue
5. pick up the cheese and continue
6. pick up the cheese and continue
7. go to the table and continue
8. put down the cheese and continue
9. done.
Human: throw away the food on the table
Robot: 1. go
to table and ask: What food items are there on the table? **Human**: There is an orange and lime soda on the table
2. find an orange and continue
3. pick up the orange and continue
4. go to the trash and continue
5. put down the orange and continue
6. find a lime soda and continue
7. pick up the lime soda, **Human**: I changed my mind. Please keep the lime soda here because I haven't finished it
8. put down the lime soda and continue
9. done.
Human: let go of the banana
Robot: 1. put down the banana and continue
2. done.
Human: bring me something healthy to eat
Robot: 1. go to
counter and ask: What food items do you have? **Human**: We have apples, bananas, multigrain chips, and kettle chips
2. find an apple and continue
3. pick up the apple and continue
4. bring it to you and continue
5. put down the apple, **Human**: I actually don't want to have fruits. Can you bring me something else?
6. find multigrain chips and continue
7. pick up multigrain chips and continue
8. bring it to you and continue
9. put down multigrain chips and continue
10. done.
Human: deliver a can of coke to me
Robot: 1. find a coke and continue
2. pick up the coke and continue
3. bring it to you and continue
4. put down the coke and continue
5. done.
Human: take out everything from the top drawer
Robot: 1. go to the top drawer and continue
2. open the drawer and ask: what items are there in the drawer? **Human**: There is a coke and rice chips
4. pick up the coke from the drawer and continue
5. put down the coke on the table and continue
7. pick up rice chips from the drawer and continue
8. put down rice chips
on the table and ask: Did I successfully take out everything from the drawer? **Human**: The coke is still there
9. pick up the coke from the drawer and continue
10. put down the coke on the table and continue
11. done.